SUMMER INTERNSHIP PRESENTATION

Smart Course Enrollment System

A centralized platform for course discovery and online enrollment requests.

PYTHON / FASTAPI / MYSQL

DHAARANYA S

Register No. 810023205019

Department of Information Technology

UCE (BIT Campus), Anna University, Tiruchirappalli

The project at a glance

From browsing a course to recording an enrollment request.

Discover

Browse learning categories and compare courses by instructor, duration, level and seats.

Enroll

Submit learner details and a preferred course through one online form.

Monitor

View summary figures, available courses and the latest enrollment requests.

Project scope: a working enrollment prototype. Secure authentication, approval workflows and live seat allocation are future enhancements.

Company & internship experience

Astonish InfoTech Private Limited, Tiruchirappalli.

Internship context

Duration: 01 June - 30 June 2026

Domain: Python full-stack development

Project: Smart Course Enrollment System

Practical learning

Requirement analysis and interface design

FastAPI routing and Jinja2 rendering

MySQL integration, testing and documentation

Why replace manual enrollment?

The report identifies a need for a single, accessible registration platform.

Scattered information

Course details spread across brochures, registers and spreadsheets.

Repeated manual work

Paper forms and re-entry increase the risk of errors and duplicate records.

Limited convenience

Office-hour dependence and waiting time make registration less convenient.

Difficult tracking

Staff must manually collect records and prepare enrollment summaries.

What the system aims to achieve

A simple learner journey supported by centralized records.

- Make categories and detailed course information easy to explore.
- Provide category-based filtering to narrow the course catalog.
- Capture enrollment requests through an online form.
- Present recent requests and summary figures on a dashboard.
- Reduce paperwork and demonstrate end-to-end full-stack development.

Technology stack

Each technology has a clear role in the application.

Interface	HTML5 + CSS3 + JavaScript	Responsive pages, cards, forms and footer year
Backend	Python + FastAPI	Routes, form processing and database operations
Templates	Jinja2	Render database content into HTML pages
Data storage	MySQL + Connector/Python	Store and retrieve categories, courses and requests
Development	Uvicorn + Visual Studio Code	Serve the application and develop the project

How the application works

Three layers separate the interface, application logic and stored data.

01 Presentation

Browser
HTML, CSS, JavaScript

Learner views pages
and submits forms

>

02 Business logic

FastAPI + Jinja2

Handle requests, run SQL
and render HTML

>

03 Database

MySQL

Categories
Courses
Enrollments

Request path: Browser -> FastAPI -> MySQL. Response path: Database results -> Jinja2 template -> HTML page in the browser.

Database design

Three core tables support discovery and request storage.

Categories

id (primary key)
name (unique)
description
image / color
created_at

Courses

id (primary key)
category_id (foreign key)
title / instructor
duration / level / seats
description / created_at

Enrollments

id (primary key)
student_name / email
course_name / category_name
phone / status
created_at

Relationship: one category has many courses. In the current prototype, enrollments store course and category names rather than foreign-key IDs.

Main application modules

The pages work together as a single enrollment platform.

Home & categories

Introduction, summary cards and category browsing

Course catalog

Detailed course listings and a category filter

Enrollment

Learner form, database insert and confirmation

Login

Basic input checks and dashboard redirection

Dashboard

Available courses and eight latest requests

Contact & setup

Support details and automatic table initialization

Learner enrollment journey

A successful submission records a request; it does not confirm a seat.

1

Browse

Open categories

2

Compare

Filter the catalog

3

Complete

Enter learner details

4

Submit

Send the online form

5

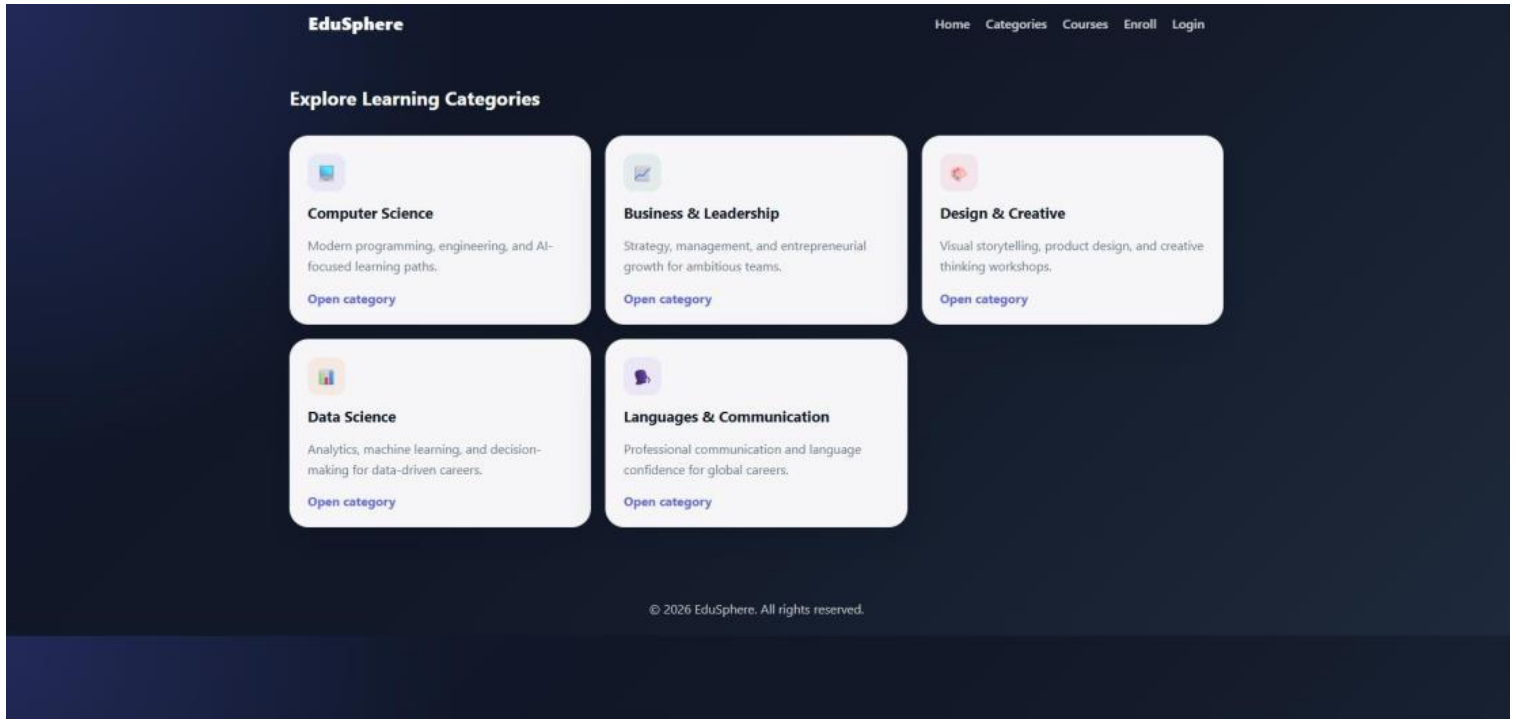
Record

Save with Pending status

Server action: read form values -> trim selected text fields -> insert enrollment into MySQL -> render the success message.

Course discovery & category filtering

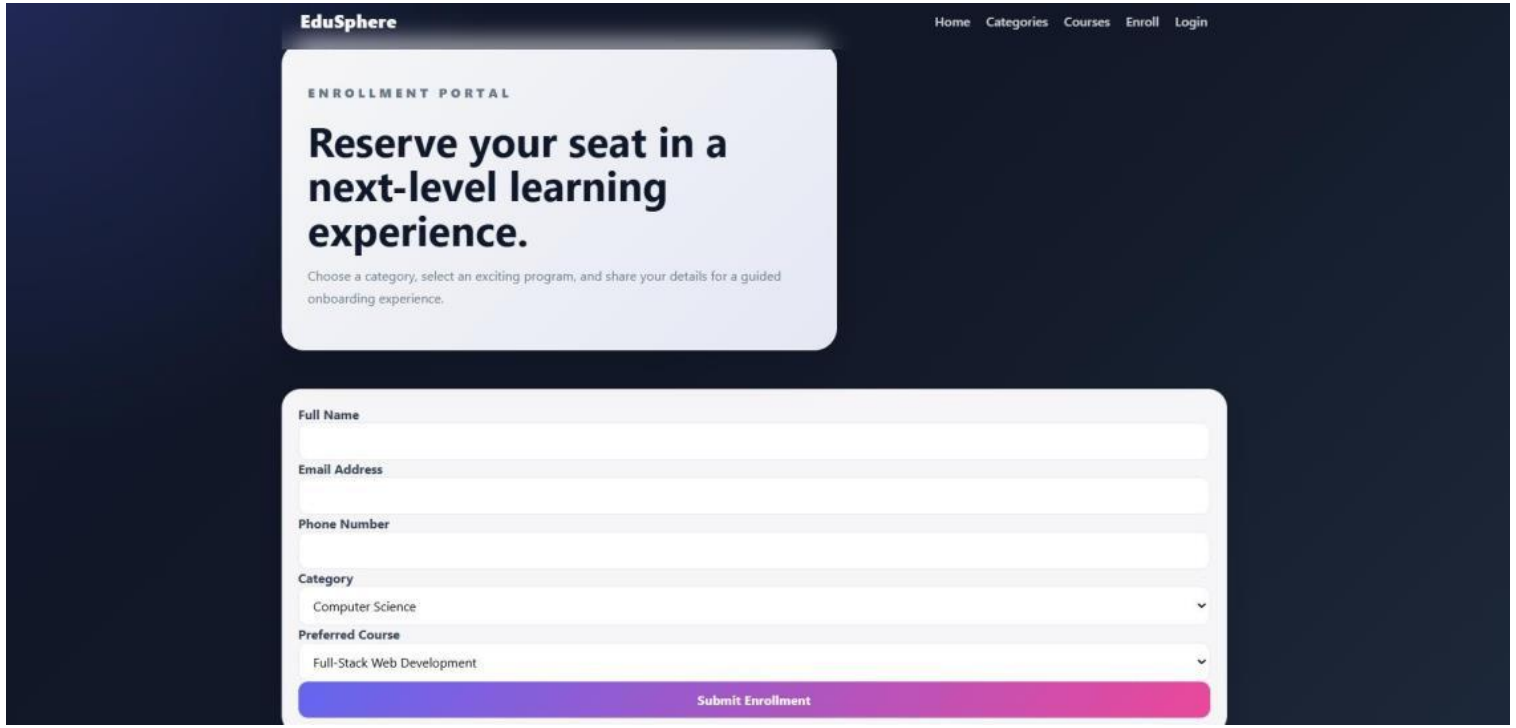
Actual course catalog screenshot from the internship report.



- Catalog joins courses with their category.
- A query parameter selects the category.
- Parameterized SQL filters matching records.
- Cards show instructor, duration, level and seats.

Online enrollment & confirmation

Actual successful-submission screenshot from the report.



- Form captures name, email, phone, category and course.
- HTML checks required fields and email format.
- The backend saves a new request in MySQL.
- A success message acknowledges receipt.

Login and dashboard: current scope

The prototype demonstrates navigation and monitoring, not secure account access.

Login behavior

Checks for a non-empty username and password.

Requires a password of at least four characters.

Redirects to the dashboard with HTTP 303.

Dashboard behavior

Shows category and course figures.

Displays available courses and the latest eight enrollment requests.

The enrollment statistic is capped at eight.

Important limitation: credentials are not verified against accounts, and the dashboard route has no authentication check in the shown code.

30-day implementation timeline

From initial requirements to a working prototype and final report.

Days 1-5	**Plan & prepare**	Requirements, scope and development setup
Days 6-10	**Design foundations**	FastAPI, Jinja2, architecture and database schema
Days 11-17	**Build the base**	UI design, project structure, database helpers and setup
Days 18-25	**Connect modules**	Categories, catalog, enrollment, login and dashboard
Days 26-30	**Test & document**	Functional checks, fixes, screenshots and presentation

Testing & reported observations

Qualitative findings from the internship report; no benchmark measurements are supplied.

Functional checks

Category browsing and course filtering

Enrollment insertion and confirmation

Empty fields, invalid email and short password

Database-unavailable scenario

Observed outcomes

Core workflow demonstrated successfully.

Card-based pages adapted to smaller screens.

Simple queries felt fast with sample data.

Central records simplified recent-request viewing.

Evidence boundary: screenshots and narrative testing support a prototype demonstration, not measured scalability or production readiness.

Known gaps in the prototype

Distinguishing implemented functions from planned improvements.

Access control	Login validates input only; dashboard access is unprotected.
Data consistency	Course and category selections are independent.
Seat availability	Displayed seat numbers do not decrease after requests.
Statistics	Enrollment count reflects at most eight recent records.
Data quality & security	Duplicate sample courses were observed; DB settings remain in code.

Future enhancements

A suggested sequence based on the improvements listed in the report.

1 Secure & validate

Hashed passwords and authenticated sessions

Role-based access

Server-side validation and environment-based settings

2 Strengthen operations

Admin approval panel

Seat allocation and waiting lists

Foreign-key enrollment links and duplicate prevention

3 Expand the platform

Email / SMS updates

Payments and analytics

Search, pagination, automated tests and cloud deployment

A practical foundation for digital enrollment

The project connects frontend design, backend logic and relational data storage.

One platform.
Clear course discovery.
Centralized enrollment requests.

Key takeaway: the system demonstrates a complete browse-to-request workflow using FastAPI, Jinja2 and MySQL, with a clear path toward secure, production-ready enrollment management.

THANK YOU

Questions & discussion

Source: Dhaaranya S, Smart Course Enrollment System, Summer Internship Report.